Tutorial 13

Questions from Problem Set 6

Question 1

Recall the following recursive algorithm that we saw in class, for solving the Rod Cutting problem. Here n is the length of the rod to be cut up and sold, and P is an array of selling prices for different lengths of rod.

CutRod(n, P)

```
1 if n == 0
2     return 0
3 maxRevenue ← −1
4 for i ← 1 to n
5     iFirstRevenue = P[i] + CutRod((n − i), P)
6     if iFirstRevenue > maxRevenue
7         maxRevenue ← iFirstRevenue
8 return maxRevenue
```

Question 1 (contd.)

(a) Explain why **CutRod** correctly solves the Rod Cutting problem.

Question 1 (contd.)

(b) Note that—assuming that each array access and each operation involving up to two numbers can be done in constant time—the running time of the algorithm is proportional to the number of times that **CutRod** is invoked, starting with the initial call CutRod(n, P). Write a recurrence for this number, and solve it to get an asymptotic estimate of this number.

Question 1 (contd.)

(c) Write pseudocode that memoizes the above recursive solution. Derive an upper bound on the running time of this memoized algorithm, as a function of n. How does this bound compare to the bound that you derived for the recursive solution in part (b)?

Question 2

We derived the following dynamic programming solution to Rod Cutting in class:

CutRod-DP(n, P)

```
1  R ← an array of length n + 1      // R = R[0 . . . n]
2  R[0] = 0
3  for j ← 1 to n
4     jMaxRevenue = −1
5     for i = 1 to j
6        iRevenue = P[i] + R[j − i]
7        if iRevenue > jMaxRevenue
8           jMaxRevenue ← iRevenue
9     R[j] ← jMaxRevenue
10 return R[n]
```

Question 2 (contd.)

We have now seen three solutions to this problem: the recursive solution given as part of Question 1, the memoized version of Question 1(c), and the above DP. Each of these algorithms only gave us the maximum revenue; they did not tell us where/how to cut the input rod to realize this maximum revenue.

Modify each of these three algorithms for Rod Cutting so that it also returns a collection of lengths (starting with zero at one end of the rod, and ending with n at the other end) where we can cut the input rod of length n to realize the maximum possible revenue. A sample output for $n = 10$ and some array P of prices might look like: "2, 4, 7, 9".

Hint: Simplify and conquer. See if it suffices to find, for each length $0 \leq i \leq n$, the best place to make the first cut on a rod of length i.

Question 3

Recall the problem of efficiently multiplying a chain of matrices that we saw in class:

Least Cost Matrix-chain Multiplication
Input: An array $D[0 \ldots n]$ of $n+1$ positive integers.
Output: The least cost (total number of arithmetic operations required) for computing the matrix product $A_1 A_2 \cdots A_n$ where each A_i; $1 \leq i \leq n$ has dimensions $D[i-1] \times D[i]$, given that multiplying a $p \times q$ matrix with a $q \times r$ matrix requires (pqr) arithmetic operations.

Let $OPT(s,t)$ denote the least ("optimum") cost for multiplying the sub-sequence $A_s, A_{s+1}, \ldots, A_t$. If $s = t$ then $OPT(s,t) = 0$.

Question 3 (contd.)

(a) Explain why there must exist an index $1 \leq i < n$ such that $OPT(1, n) = OPT(1, i) + OPT(i + 1, n) + d_0 d_i d_n$.
Note: The main thing to show is not the fact that such an i exists, but the claim that it is OK to just add together the $OPT()$ values of the sub-sequences. In particular: Why is it that the global optimum is not smaller than this sum?

Question 3 (contd.)

(b) Write the pseudocode for a recursive algorithm that solves Least Cost Matrix-chain Multiplication, using the claim proved in part (a). Argue that this algorithm correctly solves the problem.

Question 3 (contd.)

(c) Prove that your algorithm from part (b) makes $\Omega(2^n)$ recursive calls.

Question 3 (contd.)

(d) Memoize your algorithm from part (c). Show that it now runs in $\Theta(n^c)$ time for some fixed constant c. What is the value of c that you get?

Question 3 (contd.)

(e) Write the pseudocode for a non-recursive algorithm that solves this problem using dynamic programming. Argue that your procedure correctly solves the problem. What is the running time of your algorithm in the asymptotic notation? *Hint:* Try projecting onto the "breakpoint" index i of part (a).

Question 4

Consider the following two-player game.

Pots-of-Gold Game
Input: An array $G[0 \ldots n-1]$ of n positive integers.
Output: The maximum total value of gold that the first player can guarantee, assuming that both players play optimally, in the following game: There are n pots of gold arranged in a row, where pot i contains $G[i]$ gold coins. The two players alternately remove one pot from either end of the row. The first player moves first.

For integers i, j with $0 \leq i \leq j \leq n-1$, let $OPT(i, j)$ denote the maximum value of gold that the player whose turn it is to move can guarantee from the subarray $G[i \ldots j]$, assuming optimal play by both players.

Question 4 (contd.)

(a) Give the values of $OPT(i, j)$ in the base cases when $i = j$ and when $j = i + 1$.
Now consider the case when $j \geq i + 2$. The current (first) player may choose either pot $G[i]$ or pot $G[j]$. After this move, the other player will move next and, playing optimally, will attempt to minimize the amount of gold that the current player eventually obtains. Using this observation, derive a recurrence relation that expresses $OPT(i, j)$ in terms of values $OPT(i', j')$ for smaller intervals $G[i' \ldots j']$.
Explain why your recurrence correctly captures the optimal play of both players. In particular, argue why the guaranteed optimum is not smaller than the value given by your recurrence.

Question 4 (contd.)

(b) Write pseudocode for a recursive algorithm that computes $OPT(0, n-1)$ using the recurrence obtained in part (a). Argue that your algorithm correctly solves the Pots-of-Gold Game problem.

Question 4 (contd.)

(c) Let $T(n)$ denote the number of recursive calls made by your algorithm on an input of length n in the worst case. Write a recurrence for $T(n)$, and use it to show that your algorithm makes $\Omega(2^n)$ recursive calls.

Question 4 (contd.)

(d) Memoize your recursive algorithm from part (b). Explain why the number of distinct subproblems is polynomial in n, and deduce that the running time of the memoized algorithm is $\Theta(n^c)$ for some constant c. What value of c do you obtain?

Question 4 (contd.)

(e) Write pseudocode for a non-recursive algorithm that solves the Pots-of-Gold Game problem using dynamic programming. Argue that your algorithm correctly computes $OPT(0, n-1)$. What is the running time of your algorithm in asymptotic notation?

Question 5

In this question we derive a different set of algorithms for the Pots-of-Gold game. Instead of directly computing the amount obtained by the first player, we consider an alternative state.

For integers i, j with $0 \leq i \leq j \leq n-1$, let $DIFF(i, j)$ denote the maximum difference

(gold obtained by the current player)$-$(gold obtained by the opponent)

that the player whose turn it is to move can guarantee from the subarray $G[i \ldots j]$, assuming optimal play by both players.

Question 5 (contd.)

(a) Give the value of $DIFF(i, j)$ in the base case when $i = j$. Now consider the case when $j > i$. The current player may choose either pot $G[i]$ or pot $G[j]$. After the current player takes one pot, the two players' roles are reversed for the remaining subarray (the opponent now becomes the "current player"). Using this observation, derive a recurrence relation that expresses $DIFF(i, j)$ in terms of values $DIFF(i', j')$ for smaller intervals. Explain why your recurrence correctly models optimal play by both players. In particular, argue why the guaranteed difference is not smaller than the value given by your recurrence.

Question 5 (contd.)

(b) Write pseudocode for a recursive algorithm that computes $DIFF(0, n-1)$ using the recurrence obtained in part (a). Explain how the value of $DIFF(0, n-1)$ can be used to compute the maximum gold that the first player can guarantee. Using this idea, write a recursive algorithm that solves the original question. Argue that your algorithm correctly solves the Pots-of-Gold Game problem.

Question 5 (contd.)

(c) Let $T(n)$ denote the number of recursive calls made by your algorithm on an input of length n in the worst case. Write a recurrence for $T(n)$, and show that your algorithm makes $\Omega(2^n)$ recursive calls.

Question 5 (contd.)

(d) Memoize your recursive algorithm from part (b). Explain why the number of distinct subproblems is polynomial in n, and deduce that the running time of the memoized version is $\Theta(n^c)$ for some constant c. What value of c do you obtain?

Question 5 (contd.)

(e) Write pseudocode for a non-recursive algorithm that solves the Pots-of-Gold Game problem using dynamic programming based on the quantity $DIFF(i, j)$. Argue that your algorithm correctly computes $DIFF(0, n-1)$, and explain how to recover the maximum gold guaranteed for the first player. What is the running time of your algorithm in asymptotic notation?

Question 6

The 0-1 Knapsack Without Repetition problem is defined as follows:

0-1 Knapsack Without Repetition
Input: A non-negative integer n; a set of n items $\mathcal{I} = \{I_1, I_2, \ldots, I_n\}$ where item I_j has value v_j and weight w_j; and a maximum weight capacity W. The values, weights, and W are all integers.
Output: The maximum sum, taken over all subsets of $\mathcal{I}$ of total weight at most W, of the total value of the items in that set. That is, the goal is to maximize $\sum_{i=1}^{n} v_i x_i$ subject to $(\sum_{i=1}^{n} w_i x_i) \leq W$ and $x_i \in \{0, 1\}$ for $1 \leq i \leq n$.

Assume weights and values are given as arrays $Value[1 \ldots n]$ and $Weight[1 \ldots n]$.

Question 6 (contd.)

(a) Write the pseudocode for a recursive procedure **NoRepKnapSackRec** which solves the 0-1 Knapsack Without Repetition problem for these inputs. Argue that your procedure correctly solves the problem. Write a recurrence for the running time of the algorithm, and solve it to obtain a worst-case upper bound on the running time of the algorithm on these inputs.

Question 6 (contd.)

(b) Memoize your algorithm of part (a). What is the running time of this version?

Question 6 (contd.)

(c) Write the pseudocode for a non-recursive procedure **NoRepKnapSackDP** which solves the 0-1 Knapsack Without Repetition problem for these inputs in time polynomial in $(n + W)$, using dynamic programming.
Hint: Try projecting onto a prefix of the list of items. That is, for each $1 \leq i \leq n$, let projection S_i denote the set of all solutions which pick items only from the subset $\{I_1, I_2, \ldots, I_i\}$. Argue that your procedure correctly solves the problem. What is the running time of your algorithm in the asymptotic notation?

Question 7

The 0-1 Knapsack With Repetition problem is defined as follows:

0-1 Knapsack With Repetition
Input: A non-negative integer n; a set of n item types $\mathcal{T} = \{T_1, T_2, \ldots, T_n\}$ where each item of type T_j has value v_j and weight w_j; and a maximum weight capacity W. The values, weights, and W are all integers.
Output: The maximum sum, taken over all multisubsets of $\mathcal{T}$ of total weight at most W, of the total value of the items represented by that multiset.
That is, the goal is to maximize $\sum_{i=1}^{n} v_i x_i$ subject to $(\sum_{i=1}^{n} w_i x_i) \leq W$ and $x_i \in (\mathbb{N} \cup \{0\})$ for $1 \leq i \leq n$.

The difference from 0-1 Knapsack Without Repetition is that here we are allowed to pick more than one item of each type into the collection.

Question 7 (contd.)

(a) Write the pseudocode for a recursive procedure **RepKnapSackRec** which solves the 0-1 Knapsack With Repetition problem for these inputs. Argue that your procedure correctly solves the problem. Write a recurrence for the running time of the algorithm, and solve it to obtain a worst-case upper bound on the running time of the algorithm on these inputs.

Question 7 (contd.)

(b) Memoize your algorithm of part (a). What is the running time of this version? notation?

Question 7 (contd.)

(c) Write pseudocode for a non-recursive procedure **RepKnapSack1** which solves the 0-1 Knapsack With Repetition problem for these inputs, using dynamic programming. *Hint:* Try projecting onto the item types in a solution. Argue that your procedure correctly solves the problem. What is the running time of your algorithm in the asymptotic notation?

Question 7 (contd.)

(d) Write pseudocode for a non-recursive procedure **RepKnapSack2** which solves the 0-1 Knapsack With Repetition problem for these inputs, using dynamic programming. *Hint:* Try projecting onto the number of items in a solution. Argue that your procedure correctly solves the problem. What is the running time of your algorithm in the asymptotic notation?

Question 8

A subsequence of an array A is any sub-array of A, obtained by deleting zero or more elements of A without changing the order of the remaining elements. An increasing subsequence of an integer array A is a sub-sequence of A which is in strict increasing order.

The input to the **Longest Increasing Subsequence** problem consists of an integer array A, and the goal is to find an increasing subsequence of A with the most number of elements. In this question we will deal with the (slightly) simpler problem of finding the *length* of a longest increasing subsequence of the input array.

Question 8 (contd.)

(a) Write the pseudocode for a recursive algorithm **LenLis**(A, n) that takes an integer array $A[1 \ldots n]$ of length n as input and returns the length of a longest increasing subsequence of A.

   i. First, write the pseudocode for **LenLis**(A, n) assuming that you have access to a function **LenLisBigger**(A, n, i, j) which takes A, n, and two indices $1 \leq i < j \leq n$ as inputs, and returns the length of a longest increasing subsequence S of $A[j \ldots n]$ with the property that every element of S is larger than $A[i]$.

   ii. Now write recursive pseudocode for **LenLisBigger**(A, n, i, j). Make sure that you correctly deal with the base cases.

(b) Write a recurrence for the running time of your **LenLis**(A, n) function and solve it.

(c) Memoize your **LenLis**(A, n) function. Derive an upper bound on the running time of the memoized version. How does this compare with the running time of the pure recursive version?

Question 9

In this question you will design DP algorithms for the problem of finding the length of a longest increasing subsequence, as defined in the previous question.

(a) Write the pseudocode for a non-recursive procedure **LenLisDP1**(A, n) that finds the length of a longest increasing subsequence of integer array $A[1 \ldots n]$ using dynamic programming, in time polynomial in n.
Hint: Try projecting the solution space—the set of all increasing subsequences of A—onto starting indices. That is, for $1 \leq i \leq n$ let S_i denote all those solutions whose first element is $A[i]$.
Argue that your procedure correctly solves the problem. What is the running time of your algorithm in the asymptotic notation?

Question 9 (cont.)

(b) Write the pseudocode for a non-recursive procedure **LenLisDP2**(A, n) that finds the length of a longest increasing subsequence of integer array $A[1 \ldots n]$ using dynamic programming, in time polynomial in n.
Hint: Try projecting the solution space onto ending indices. That is, for $1 \leq i \leq n$ let S_i denote all those solutions whose last element is $A[i]$.
Argue that your procedure correctly solves the problem. What is the running time of your algorithm in the asymptotic notation?

Question 10

Let A, B be two arrays. An array X is said to be a *common subsequence* of A and B if X is a subsequence both of A and of B. The input to the **Longest Common Subsequence** problem consists of two arrays A and B, and the goal is to find a common subsequence of A and B with the most number of elements. In this question we look at the (slightly) simpler problem of finding the *length* of a longest common subsequence of the input arrays.

Question 10 (cont.)

(a) Write the pseudocode for a non-recursive procedure that finds the length of a longest common subsequence of input arrays $A[1 \ldots m]$ and $B[1 \ldots n]$ using dynamic programming, in time polynomial in $(m + n)$.
Hint 1: Try projecting the solution space onto starting indices. That is, for $1 \leq i \leq m$, $1 \leq j \leq n$ let $S_{i,j}$ denote all those common subsequences whose first elements in the two arrays are $A[i]$, $B[j]$, respectively.
Hint 2: Try projecting the solution space onto ending indices. That is, for $1 \leq i \leq m$, $1 \leq j \leq n$ let $S_{i,j}$ denote all those common subsequences whose last elements in the two arrays are $A[i]$, $B[j]$, respectively.

(b) Argue that your procedures correctly solve the problem.

(c) What are the running times of your algorithms in the asymptotic notation?

Question 11

The input to this problem is an $m \times n$ array $C[0 \ldots (m-1)][0 \ldots (n-1)]$ of positive integers. A *valid path* through this array is a path that starts at location $C[0][0]$ and ends at location $C[m-1][n-1]$ using (only) the following three types of steps: (i) move to the right by one cell, (ii) move down by one cell, and (iii) move diagonally down and to the right by one cell. That is, if the path is currently at location $C[i][j]$ then after one step it can be only at one of the following three locations: (i) $C[i][j+1]$; $j < n$, (ii) $C[i+1][j]$; $i < m$, and (iii) $C[i+1][j+1]$; $i < m$, $j < n$. The cost of a valid path is the sum of all the array elements which the path touches, including the first and last elements in the path.

The goal is to compute the *least cost* of a valid path through the input array C.

Question 11 (contd.)

(a) Write the pseudocode for a non-recursive procedure that takes C, m, n as arguments and finds the least cost of a valid path through C using dynamic programming, in time polynomial in $(m + n)$.
Hint 1: Try projecting the solution space—the set of all valid paths through C—onto the direction of the first step that the path takes.
Hint 2: Try projecting the solution space onto the direction of the last step that the path takes.

(b) Argue that your procedures correctly solve the problem.

(c) What are the running times of your algorithms in the asymptotic notation?

Question 12

The input to this problem consists of an array $C[0 \ldots (n-1)]$ of n distinct positive integers, and a target positive integer T. The elements of C are coin denominations, and T is a target amount to be made up using these denominations. The goal is to find the least number of coins, of these denominations, that can be used to make up the amount T. Note that each denomination may be used any number of times. The goal is to minimize the total number of coins, not of denominations, used. If a certain denomination $C[i]$ is used d times, this adds d to the number of coins.

Question 12 (cont.)

(a) Write the pseudocode for a non-recursive algorithm that accepts C, n, T as arguments and computes the smallest number of coins of the denominations specified by C, that can be used to make up the target amount T. If the given denominations cannot be used to make up the specified target—Example: $C = [2, 4, 6]$, $T = 15$—then the algorithm should return **False**. The algorithm should solve this problem using dynamic programming, in time polynomial in $(n + T)$.
Hint 1: Try projecting onto the coin denominations in a solution.
Hint 2: Try projecting onto the number of coins in a solution.

(b) Argue that your procedures correctly solve the problem.

(c) What are the running times of your algorithms in the asymptotic notation?

Question 13

Climbing Stairs

You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Question 14

Min Cost Climbing Stairs

You are given an integer array cost where cost[i] is the cost of ith step on a staircase. Once you pay the cost, you can either climb one or two steps.

You can either start from the step with index 0, or the step with index 1.

Return the minimum cost to reach the top of the floor.

Question 15

Word Break

Given a string s and a dictionary of strings wordDict, return true if s can be segmented into a space-separated sequence of one or more dictionary words.

Note that the same word in the dictionary may be reused multiple times in the segmentation.

Question 16

Integer Replacement

Given a positive integer n, you can apply one of the following operations:

- If n is even, replace n with $n/2$.
- If n is odd, replace n with either $n + 1$ or $n - 1$.

Return the minimum number of operations needed for n to become 1.

Question 17

Partition Array for Maximum Sum

Given an integer array arr, partition the array into (contiguous) subarrays of length at most k.

After partitioning, each subarray has the values of elements changed to become the maximum value of that subarray.

Return the largest sum of the given array after partitioning.

Questions from Handout

Non-Recursive DP

Warm Up: Blackbird's Ship

Ace has been captured by Blackbeard and is held prisoner aboard the Saber of Xebec, a massive ship with n cabins. Blackbeard is demanding a hefty ransom.

Nami, negotiating on behalf of the Straw Hats, agreed to the following terms. She will guess the cabin number where Ace is held. If she guesses correctly, Blackbeard releases Ace immediately. Otherwise, if her guess is too high, Blackbeard informs her of this and she pays him a Berries. If her guess is too low, she pays b Berries.

We want to compute the minimum cost Nami can incur using the optimal strategy, irrespective of where Ace is imprisoned. Can you find an $O(n)$ algorithm to compute this?

Hint: Consider $dp[l]$ that represents the maximum sized ship we can search at cost of l berries.

Travelling Rockstar

Given n cities and a table T such that $T[i][j]$ is time it takes to go from city i to city j.

A rockstar wants to have a tour covering all the n cities. Can you find a tour that minimizes the travel time?

Note: We want a $O(n^2 \cdot 2^n)$ algorithm.

All Pairs Shortest Path

Given a weighted graph $G = (V, E)$, design an algorithm to compute the shortest path distances between every pair of vertices in V.

Note: We would like a $O(|V|^3)$ algorithm.

Hint: The algorithm is dumber than you could think of. It is like so, so dumb!

Single Source Shortest Path

Given a weighted graph with positive weights $G = (V, E)$, a start location S and target location T; design an algorithm to find the shortest path from S to T.

Note: This algorithm's story is like one of the coolest thing I know of.

Party

You are organizing a party for your office. Unfortunately, the office is very hierarchical and people don't feel comfortable attending a party if their immediate senior is attending (although they have no issues with others. Basically, you and boss of your boss can be invited but not you and your boss).

Given a list S of length n with $S[i]$ being the immediate underlings of i ($S[j]$ being empty indicates j is at the bottom of the hierarchy). Everyone has only one immediate boss.

What is the most number of people you can invite?

Hint: We want a $O(n)$ algorithm.

Party with Money

Same setup as Party but we need money to even do this party. Every person has an amount of money they are willing to contribute. We want to invite the people, in an attempt to maximize the amount of money we have to party. Can you do that in $O(n)$?

Root Replace DP

Given a tree with n nodes, find a node such that when this node is the root, the sum of the depths of all nodes is maximized.

Note: This can be done in $O(n)$ time as well.

Digital DP

Given positive integers a and b, how many times does each digit appear in all integers $[a, b]$?

Hint: We can solve this in $O(\log_{10}(b))$.

Hint: $9000 \to 9900 \to 9990 \to 9999$ also seems like a valid way to count... doesn't it?

Just a Normal DP

Given a positive integer n, determine how many ways it can be expressed as the sum of k positive integers, where different orders are considered distinct partitions.

Another Normal DP

Starting with an empty list [], we have two operations:

- Increase all the elements with 1
- Append 1 to the list

Given an n, what is the minimum number of operations we need to make the sum of the list n?

Oh, That's Why They Were There!

Given a positive integer n, how many ways are there to partition n into a sum of any number of positive integers? (Different orders are considered the same partition.)

Hint: A strategy similar to *Just a normal DP* will give an $O(n^2)$ solution. Similarly, *Another normal DP* might also provide a $O(n^2)$ solution.

Extension: Looking at the solutions, can you see some part of the solution that uses more time in both? Notice that these are both disjoint. So maybe we can combine both the solutions? This would give an $O(n\sqrt{n})$ solution.

0-1 Knapsack

Given a bag with capacity C and N items such that the weight of i-th item is $W[i]$ and value is $V[i]$.

Choose a subset I of $[1, N]$ that maximizes:

$$\sum_{i \in I} V[i]$$

up to the constraint

$$\sum_{i \in I} W[i] \leq C$$

Note: We have seen a solution that took $O(NC)$ space last time. Can you find a solution taking only $O(C)$ space?

Unbounded Knapsack

The model is same as 0-1 Knapsack with the change that we can choose each item an arbitrary number of times. To state formally,

Choose a multiset I of $[1, N]$ that maximizes:

$$\sum_{i \in I} V[i]$$

up to the constraint

$$\sum_{i \in I} W[i] \leq C$$

Note: We wish to find a $O(NC)$ time and $O(C)$ space solution.

Bounded Knapsack

The model is same as 0-1 Knapsack but now for the i-th item, there is a number $B[i]$ that is the stock of item i or how many times you can add it to the bag.

Formally, choose a multiset I of $[1, N]$ that maximizes $\sum_{i \in I} V[i]$ up to the constraints

$$\sum_{i \in I} W[i] \leq C \quad \text{and} \quad \forall i \in N,\ |\{x = i \mid x \in I\}| \leq B[i]$$

Note: We want a $O(CN \log(\max(B)))$ algorithm. There is an obvious $O(C \cdot \text{sum}(B)) = O(CN \cdot \max(B))$ algorithm which you could start with.

Mixed Knapsack

Let's say we have all the above types of items. Can you find a way to solve this efficiently?

Grouped Knapsack

Let's go back to 0-1 Knapsack.

This time there are groups of items and we can take only one from each group. Can you find an efficient algorithm that solves this?